

Day 1 SDE-2 Interview Guide: Spring Core & Persistence

Target Roles: SDE-2 / Senior Backend | Project: Digital Wallet & Payment Ledger

Track: Core Spring Internals & Domain Persistence

Part 1 — Spring Core Internals & Bean Lifecycle

Q1. What is the difference between BeanFactory and ApplicationContext? Why does Spring Boot use ApplicationContext?

Simple Answer

Both BeanFactory and ApplicationContext are **Spring IoC containers**.

Their job is to create and manage Spring beans.

BeanFactory

BeanFactory is the basic Spring container.

It mainly provides:

- Creating beans
- Dependency Injection
- Managing bean lifecycle

It generally supports **lazy creation** of beans.

Example:

```
BeanFactory factory = ...;
```

```
PaymentService service =
```

```
    factory.getBean(PaymentService.class);
```

The bean can be created when it is requested.

ApplicationContext

ApplicationContext is a more powerful version of BeanFactory.

It provides everything from BeanFactory plus features like:

- Application events
- AOP
- Internationalization
- Automatic handling of many infrastructure features
- Eager creation of singleton beans by default

Spring Boot normally uses ApplicationContext.

Why?

Because in a real application, we want configuration problems to be detected **when the application starts**, rather than when the first customer makes a payment.

For example:

Application starts



Spring creates beans



Missing dependency found



Application fails immediately

Instead of:

Application starts successfully



Customer makes payment



PaymentService requested



Missing dependency discovered



Payment request fails ❌

Interview Answer

"BeanFactory is the basic IoC container, while ApplicationContext is a more feature-rich container built on top of it. Spring Boot uses ApplicationContext because it provides enterprise features such as AOP, events, and automatic bean management, and it generally initializes singleton beans at startup so configuration problems can be detected early."

Remember

BeanFactory = Basic

ApplicationContext = Full-featured Spring container

Q2. Explain the Spring Bean Lifecycle.

This is a very important interview question.

Suppose we have:

@Service

```
public class PaymentService {  
}
```

What happens when Spring starts?

Step 1 — Spring finds the class

Spring scans the application.

It sees:

```
@Service
```

```
public class PaymentService
```

Spring says:

"This is a Spring bean."

It creates metadata called a **BeanDefinition**.

Step 2 — Object is created

Spring creates the actual Java object:

```
PaymentService paymentService =
```

```
    new PaymentService();
```

Conceptually, that's what happens.

Step 3 — Dependencies are injected

Suppose:

```
@Service
```

```
public class PaymentService {
```

```
    private final PaymentRepository repository;
```

```
    public PaymentService(PaymentRepository repository) {
```

```
        this.repository = repository;
```

```
    }
```

```
}
```

Spring first creates PaymentRepository.

Then gives it to PaymentService.

```
PaymentRepository
```

↓

```
PaymentService
```

Step 4 — Aware interfaces

If the bean implements interfaces such as:

```
BeanNameAware
```

```
ApplicationContextAware
```

Spring provides that information to the bean.

For example:

BeanNameAware

↓

"paymentService"

The bean can know its Spring bean name.

You don't normally need these in everyday development.

Step 5 — Before initialization

Spring executes:

postProcessBeforeInitialization()

through BeanPostProcessor.

Think:

"Spring gets a chance to inspect/modify the bean before initialization finishes."

Step 6 — Initialization

Methods such as:

@PostConstruct

can execute here.

Example:

@PostConstruct

```
public void init() {  
    System.out.println("PaymentService initialized");  
}
```

Then initialization callbacks such as:

afterPropertiesSet()

can run, followed by a configured custom initMethod if one exists.

Step 7 — After initialization

Spring executes:

postProcessAfterInitialization()

This is an important stage because Spring AOP infrastructure can create a **proxy** around a bean when required.

For example:

@Transactional

```
public void transfer() {  
}
```

Spring may return a proxy that intercepts calls to this method.

Conceptually:

Your PaymentService



Spring Proxy



PaymentService

Step 8 — Bean is ready

Now the bean is ready to handle application requests.

Controller



PaymentService



Repository

Step 9 — Application shuts down

When Spring shuts down, destruction callbacks can run:

@PreDestroy

Then other destruction callbacks/custom destroy methods may execute where configured.

Easy lifecycle to remember

Bean Definition



Create Object



Inject Dependencies



Aware callbacks



Before Initialization



@PostConstruct



After Initialization / Proxy



Bean Ready

↓

Application Shutdown

↓

@PreDestroy

Interview Answer

"Spring first discovers bean definitions through component scanning or configuration. It then instantiates the bean, injects its dependencies, invokes aware callbacks, runs BeanPostProcessors before and after initialization, executes initialization callbacks such as @PostConstruct, and finally makes the bean available for use. During application shutdown, destruction callbacks such as @PreDestroy are executed."

Q3. What is BeanPostProcessor? How does it help @Transactional and @Async?

Don't make this complicated.

A BeanPostProcessor is basically a **Spring hook that allows Spring to process a bean before and after initialization.**

It has two important methods:

postProcessBeforeInitialization()

and

postProcessAfterInitialization()

Why is this useful?

Spring needs to add additional behavior to some beans.

For example:

@Transactional

```
public void transferMoney() {  
}
```

You wrote only the business method.

But Spring needs to do something like:

Start transaction

↓

Run transferMoney()

↓

Commit transaction

If something fails:

Start transaction

↓

Run transferMoney()

↓

Exception

↓

Rollback

Spring's AOP infrastructure uses bean post-processing to create/wire these proxies/interceptors.

Similarly:

@Async

```
public void sendEmail() {  
}
```

Spring can arrange for the method to execute asynchronously through its proxy/interceptor infrastructure.

Simple picture

Your Bean

↓

BeanPostProcessor

↓

Spring Proxy

↓

Extra behavior

↓

Your actual method

Interview Answer

"BeanPostProcessor is a Spring extension point that allows Spring to process beans before and after initialization. Spring's AOP infrastructure uses this mechanism to apply cross-cutting behavior such as transaction management and asynchronous execution, often by creating a proxy around the bean."

Q4. Why does this.credit() not trigger @Transactional or @Async?

This is a very common interview question.

Suppose:

@Service

```
public class WalletService {
```

```
    @Transactional
```

```
    public void credit() {  
    }  
}
```

```
    public void process() {  
        this.credit();  
    }  
}
```

```
}  
}
```

You might think:

```
process()
```

↓

```
credit()
```

↓

@Transactional works

But normally it doesn't.

Why?

Because Spring uses a **proxy**.

Normally the call looks like:

```
Controller
```

↓

```
Spring Proxy
```

↓

```
WalletService
```

The proxy sees:

```
@Transactional
```

and starts the transaction.

But when you do:

```
this.credit();
```

you're calling the method directly on the current object.

```
WalletService
```

↓

```
this.credit()
```

↓

```
Direct call
```

The Spring proxy is bypassed.

Therefore the transaction/asynchronous interceptor isn't invoked.

Example

```
@Service
```

```
public class WalletService {
```

```
    public void processPayment() {
```

```
    this.credit(); // proxy bypassed
}
```

```
@Transactional
public void credit() {
}
}
```

Better solution

The cleanest solution is usually to move the transactional operation into another Spring bean:

PaymentService

↓

WalletService.credit()

↓

Spring Proxy

↓

@Transactional

Other approaches exist, but for an SDE-2 interview, remember:

Self-invocation bypasses the Spring AOP proxy.

Interview Answer

"@Transactional and @Async are normally applied through Spring AOP proxies. When a method calls another method using this.method(), the call stays inside the same object and bypasses the proxy. Therefore the interceptor is not executed. A common solution is to move the method into another Spring-managed service so the call goes through the proxy."

Q5. Why is Constructor Injection preferred over Field Injection?

Suppose you write:

@Autowired

private PaymentRepository repository;

This is **field injection**.

It works, but constructor injection is generally preferred.

Instead:

private final PaymentRepository repository;

```
public PaymentService(PaymentRepository repository) {
    this.repository = repository;
}
```

Or with Lombok:

@RequiredArgsConstructor

@Service

```
public class PaymentService {
```

```
    private final PaymentRepository repository;
```

```
}
```

Reason 1 — Dependencies are obvious

With constructor injection:

```
PaymentService(PaymentRepository repository)
```

we immediately know:

PaymentService requires PaymentRepository.

Reason 2 — final

We can write:

```
private final PaymentRepository repository;
```

So the dependency cannot be reassigned.

Reason 3 — Testing is easier

We can simply do:

```
PaymentRepository repository = mock(...);
```

```
PaymentService service =
```

```
    new PaymentService(repository);
```

No Spring container is required.

Reason 4 — Circular dependencies become obvious

Suppose:

Service A → Service B

Service B → Service A

With constructor injection, Spring detects the circular dependency during startup and fails.

That's generally good because the problem is detected early.

Interview Answer

"I prefer constructor injection because it makes dependencies explicit, allows them to be final, improves unit testing, and helps Spring detect circular dependencies early. It also avoids hidden dependencies that field injection can create."

Part 2 — Persistence & Financial Domain

Q6. Why shouldn't we use float or double for money?

This one is extremely important for our Wallet project.

Suppose:

`double amount = 0.1 + 0.2;`

You might expect:

0.3

But you can get:

0.30000000000000004

Why?

Because double stores numbers using **binary floating-point representation**.

Some decimal numbers cannot be represented exactly in binary.

Why is that bad for money?

Imagine doing thousands or millions of calculations:

Transaction 1

Transaction 2

Transaction 3

...

Transaction 1,000,000

Small rounding errors can accumulate.

For a financial system, even a small difference can cause reconciliation problems.

So we use BigDecimal

Our Wallet uses:

`private BigDecimal balance;`

And:

`new BigDecimal("100.50");`

rather than:

`new BigDecimal(100.50);`

The String constructor is preferred when creating an exact decimal amount from text.

Alternative

Some financial systems store money as **integer minor units**.

For example:

₹100.50

could be stored as:

10050 paise

But in our project, we're using:

BigDecimal

Interview Answer

"float and double use binary floating-point representation, so many decimal values cannot be represented exactly. This can introduce precision errors in financial calculations. For our wallet system, we use BigDecimal with a defined scale and rounding policy so monetary calculations are predictable and precise."

Q7. Why HALF_EVEN instead of HALF_UP?

This is easier if you remember one thing:

HALF_UP

Exact .5 goes upward.

1.5 → 2

2.5 → 3

3.5 → 4

HALF_EVEN

The halfway value goes toward the nearest **even** number.

1.5 → 2

2.5 → 2

3.5 → 4

4.5 → 4

Why?

Because if you perform huge numbers of rounding operations, always rounding .5 upward can introduce a small systematic upward bias.

HALF_EVEN tries to reduce that cumulative bias.

But important!

Don't say:

"All payment systems must use HALF_EVEN."

That's incorrect.

The correct rounding mode depends on the **business/accounting/regulatory requirements**.

Interview Answer

"HALF_EVEN is useful when we want to reduce cumulative rounding bias over many calculations. Unlike HALF_UP, which always rounds exact halfway values upward, HALF_EVEN rounds halfway values toward the nearest even digit. However, the correct rounding mode ultimately depends on the financial and business requirements."

Q8. Why use Java Records for Value Objects like Money and DTOs?

We created:

```
public record Money(  
    BigDecimal amount,  
    Currency currency  
) {  
}
```

A Record is useful when an object mainly represents **data that should not be changed**.

For example:

Money

```
├── amount = 500  
└── currency = INR
```

Once created:

Money money =

```
    new Money(  
        new BigDecimal("500"),  
        Currency.INR  
    );
```

we don't have:

```
money.setAmount(...)
```

The components are final.

Records automatically provide

constructor

get-like accessors

equals()

hashCode()

toString()

For a record:

```
money.amount()
```

```
money.currency()
```

instead of:

```
money.getAmount()
```

We can also put validation inside the Record

For example:

```
public record Money(  
    BigDecimal amount,  
    Currency currency  
) {  
  
    public Money {  
        Objects.requireNonNull(amount);  
        Objects.requireNonNull(currency);  
    }  
}
```

So Records are not only for DTOs.

They can also represent **immutable value objects** such as:

Money

Address

Coordinates

DateRange

Interview Answer

"Java Records are useful for immutable data carriers such as DTOs and value objects. They reduce boilerplate by automatically providing the constructor, accessors, equals, hashCode and toString. We can also use the compact constructor to enforce invariants such as null checks or normalization."

Q9. Why did we add @Version to Wallet?

Our entity has:

```
@Version
```

```
private Long version;
```

This is for **optimistic locking**.

Let's use our wallet.

Current balance:

```
₹10,000
```

```
version = 0
```

Now imagine two requests come at almost the same time.

Request A	Request B
↓	↓
Reads ₹10,000	Reads ₹10,000
version 0	version 0

Both want to debit:

₹6,000

Without proper concurrency control, both could calculate:

$10,000 - 6,000 = 4,000$

and one update could overwrite another.

This is called a **lost update** problem.

With @Version

Hibernate uses the version when updating.

Conceptually:

UPDATE wallets

SET balance = ?,

version = version + 1

WHERE id = ?

AND version = ?

Request A updates first:

balance = ₹4,000

version = 1

Request B still thinks:

version = 0

Its update looks for:

WHERE version = 0

But the database now has:

version = 1

So the update affects zero rows.

Hibernate detects the conflict and throws an optimistic locking exception.

Important

@Version doesn't magically retry the transaction.

It **detects the conflict**.

Your application can then decide whether to retry, reload, or return a conflict response.

Interview Answer

"@Version enables optimistic locking in JPA. It helps detect lost updates when multiple transactions modify the same wallet concurrently. Hibernate includes the current version in the update condition and increments it after a successful update. If another transaction has already changed the row, the update fails and Hibernate raises an optimistic locking exception."

Q10. Why did the application fail with SchemaManagementException: missing table [wallets]?

Our configuration contains:

spring:

jpa:

hibernate:

ddl-auto: validate

The important word is:

validate

It means:

"Hibernate, check whether my Java entity mapping matches the database schema. But don't create or modify the database."

Our Entity says:

@Entity

@Table(name = "wallets")

public class Wallet {

}

So Hibernate expects:

Database

↓

wallets table

But suppose PostgreSQL has:

wallet_db

↓

No wallets table ❌

When Spring Boot starts:

Hibernate

↓

Check database

↓

wallets table?

↓

NO ❌

↓

Application startup fails

That's why you saw:

SchemaManagementException

missing table [wallets]

What does ddl-auto mean?

Common values:

validate

Check schema

Don't change schema

Good for production.

update

Try to modify database schema automatically

Convenient during development, but generally not preferred for controlled production schema management.

create

Create schema

Can destroy/recreate existing structures depending on the configuration/lifecycle.

create-drop

Creates schema and drops it when the application/session ends.

Usually useful for temporary testing, not production.

What do production applications normally use?

Usually:

ddl-auto: validate

or:

ddl-auto: none

And database changes are handled through migration tools such as:

Flyway

or:

Liquibase

For example:

Developer changes Wallet entity

↓

Create migration



Flyway



PostgreSQL



ALTER TABLE / CREATE INDEX / etc.

Interview Answer

"ddl-auto: validate tells Hibernate to validate that the database schema matches the entity mappings without creating or modifying tables. If an expected table such as wallets is missing, application startup fails. In production, schema changes are normally managed through version-controlled migration tools such as Flyway or Liquibase, while Hibernate is commonly configured with validate or none."

Day 1 — 10 Questions Quick Revision

Before an interview, read this section quickly.

1. BeanFactory vs ApplicationContext

BeanFactory

→ Basic IoC container

ApplicationContext

→ BeanFactory + enterprise features

Spring Boot uses ApplicationContext.

2. Bean Lifecycle

Scan



BeanDefinition



Instantiate



Dependency Injection



Aware



BPP Before



@PostConstruct

↓

BPP After / Proxy

↓

Ready

↓

@PreDestroy

3. BeanPostProcessor

Allows Spring to process beans before/after initialization and is used by Spring infrastructure to apply behaviors such as AOP-based transactions and async execution.

4. Self Invocation

this.credit();

bypasses the Spring proxy.

Therefore:

@Transactional ❌

@Async ❌

doesn't get applied through the proxy for that self-call.

5. Constructor Injection

Remember:

Explicit dependencies

+

final fields

+

Easy testing

+

Early circular dependency detection

6. Money

float/double ❌

BigDecimal ✅

Because financial calculations require predictable decimal precision.

7. HALF_EVEN

1.5 → 2

2.5 → 2

3.5 → 4

4.5 → 4

Reduces cumulative rounding bias.

8. Records

Immutable

+

Less boilerplate

+

equals/hashCode/toString

+

Can contain validation/behavior

9. @Version

Optimistic Locking

↓

Detect concurrent updates

↓

Prevent silent lost updates

10. ddl-auto: validate

Check Entity ↔ Database

↓

Mismatch?

↓

Application fails at startup

It does NOT create the table.

★ Most important SDE-2 mindset

When answering these questions, don't just say "**what it is.**"

Explain:

WHAT is it?

↓

WHY do we use it?

↓

WHAT problem does it solve?

↓

HOW does it work?

↓

Example from our project

For example, don't simply say:

"@Version is optimistic locking."

Say:

"@Version provides optimistic locking. We use it on our Wallet entity because multiple payment requests can modify the same wallet concurrently. Hibernate includes the version in the update condition, so if another transaction has already modified the wallet, the second update detects the conflict instead of silently overwriting the first update."